SIMATS ENGINEERING
Department of Computer Science and Engineering
ASSESSMENT TOOL 2 – COMPARATIVE ANALYSIS

Course Code:	CSA12	**Course Title:**	Computer Architecture
CO Assessed:	CO4 – Precision (BL4)	**Assessment:**	Comparative Analysis
Weightage:	30%	**Total Marks:**	100
CO4 Statement:	Implement hierarchical memory systems by differentiating cache levels (L1, L2, L3), virtual memory with paging, and advanced memory technologies (DRAM, SRAM, NAND Flash, MRAM, RRAM) based on latency, bandwidth, and throughput characteristics.		
Student Name:	M.Koushik Reddy	**Reg. No.:**	192372185
Section / Batch:	CSE-AI	**Date:**	25-08-2026

Instructions to Students

- Identify the relevant concepts of L1, L2, and L3 cache, SRAM, DRAM, NAND Flash, MRAM, RRAM, virtual memory, paging, latency, bandwidth, and throughput.
- Analyze the memory requirements of different computer systems by considering cache levels, memory technologies, access latency, bandwidth, throughput, capacity, and energy consumption.
- Evaluate the performance characteristics of different memory technologies and determine their suitability for cache memory, main memory, secondary storage, and future hierarchical memory systems.
- Compute relevant performance measures such as access time, latency, bandwidth, throughput, hit/miss rate, average memory access time (AMAT), speedup, and energy efficiency wherever applicable.
- Interpret the results by assessing memory bottlenecks, performance improvements, technology trade-offs, and the suitability of each memory technology or memory hierarchy for a given application.

QUESTIONS

1. Analyze the impact of cache hit rate and miss rate on processor performance. Assess how improving cache locality and reducing cache misses can enhance memory access efficiency and overall execution time.
2. Evaluate the role of memory latency, bandwidth, and throughput in determining system performance. Analyze how variations in these parameters affect applications with different memory-access requirements.
3. Assess the effectiveness of hierarchical memory organization using registers, cache, main memory, and secondary storage. Analyze how differences in access speed, capacity, and cost influence the design of an efficient memory hierarchy.
4. Examine the role of virtual memory and paging in supporting applications that require more memory than the available physical RAM. Analyze the effects of page faults, page replacement, and paging overhead on system latency and throughput.
5. Evaluate different memory technologies for high-performance applications based on latency, bandwidth, throughput, capacity, energy consumption, and endurance. Recommend a suitable memory hierarchy for a selected application and justify the choice based on its performance requirements.

Code of Conduct

I M.Koushik Reddy/192372185 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: M.Koushik Reddy

Marks Evaluation:

Q. No	Scope of Items **(20)**	Comparison Depth **(25)**	Use of Evidence **(20)**	Critical Thinking **(20)**	Clarity of Presentation **(15)**	**Total (out of 100)**
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 20	**/ 25**	**/ 20**	**/ 20**	**/ 15**	**/ 100**

Course Coordinator Faculty Incharge:_____

Table of Contents

1. Cache Hit Rate, Miss Rate, and Processor Performance

A cache's effectiveness is measured primarily through its hit rate (the fraction of memory references satisfied directly from the cache) and its complementary miss rate. Because cache access is typically an order of magnitude faster than main memory access, even small changes in hit rate produce large changes in the Average Memory Access Time (AMAT), which in turn governs overall processor throughput.

1.1 The AMAT Model

Average Memory Access Time is expressed as:

$$AMAT = Hit\ Time + (Miss\ Rate \times Miss\ Penalty)$$

This formula shows that performance degrades not linearly but multiplicatively with miss rate, because every miss incurs the full penalty of fetching a line from a slower level of the hierarchy (often 100–300 processor cycles for a main-memory access). A processor stalled waiting for data cannot retire instructions, so cache misses translate directly into lost cycles and reduced instructions-per-cycle (IPC).

1.2 Impact of Improving Hit Rate

- **Execution time:** Reduced stall cycles: fewer misses mean fewer pipeline bubbles, which raises effective IPC without any change in clock frequency.
- **Memory bus utilization:** A higher hit rate lowers contention on the memory bus, freeing bandwidth for other cores, DMA transfers, or prefetch requests in multi-core systems.
- **Sensitivity:** Because miss penalty typically dwarfs hit time, even a 1–2% improvement in hit rate can yield a disproportionately large drop in AMAT — a phenomenon sometimes called the 'knee' of the cache performance curve.

1.3 Improving Cache Locality to Reduce Misses

Cache misses are conventionally classified into three types, and each responds to a different optimization strategy:

Miss Type	Cause	Mitigation Strategy
Compulsory (cold)	First-ever reference to a block	Prefetching, larger block size
Capacity	Working set exceeds cache size	Loop tiling/blocking, larger cache, better data structure layout
Conflict	Multiple addresses map to the same set	Higher associativity, address padding, victim caches

Improving spatial locality (accessing nearby memory addresses together, e.g., row-major traversal of arrays matching storage order) and temporal locality (reusing recently accessed data, e.g., loop blocking/tiling for matrix multiplication) directly reduces capacity and conflict misses. Techniques such as data structure padding, array-of-structures versus structure-of-arrays redesign, and compiler-driven loop transformations (blocking, fusion, interchange) are widely used to reshape access patterns so they align with cache line and set boundaries.

1.4 Overall Effect on Execution Time

Since CPU time can be modeled as Instruction Count × CPI × Clock Cycle Time, and memory stalls are a major component of CPI in modern superscalar processors, reducing miss rate effectively reduces the memory-stall component of CPI. The net effect is faster program execution without any change to the underlying instruction set or clock speed — making cache optimization one of the highest-leverage, lowest-cost performance techniques available to both hardware designers and software developers.

2. Memory Latency, Bandwidth, and Throughput

System performance depends not just on raw processor speed but on how quickly and how much data can move between memory and the processor. Three related but distinct metrics describe this: latency, bandwidth, and throughput.

2.1 Definitions

- **Latency:** The time delay between issuing a memory request and receiving the first byte of data (measured in nanoseconds or cycles). Latency is dominated by physical signal propagation, DRAM row activation, and queuing delay.
- **Bandwidth:** The maximum rate at which data can be transferred, usually expressed in GB/s. Bandwidth depends on bus width, clock frequency, and the number of parallel channels.
- **Throughput:** The actual, sustained rate of useful data delivered under real workload conditions — always less than or equal to peak bandwidth due to overhead, access patterns, and contention.

2.2 Differing Impact by Workload Type

Application Class	Dominant Requirement	Effect of Parameter Variation
Pointer-chasing / linked structures (graph traversal, hash tables)	Low latency	Bandwidth increases matter little; each access must complete before the next dependent

Application Class	Dominant Requirement	Effect of Parameter Variation
		access can issue, so latency directly limits speed.
Streaming workloads (video processing, large matrix ops, HPC)	High bandwidth	Latency is hidden by prefetching and pipelining; throughput scales almost linearly with available bandwidth.
Transactional / database systems	Balanced latency and throughput	Sensitive to both — many small concurrent random accesses need low latency, while log/bulk writes need bandwidth.
Real-time / embedded control	Predictable (bounded) latency	Worst-case latency matters more than average throughput; jitter can cause deadline misses.

2.3 Analysis

A system can be bandwidth-rich yet latency-poor, or vice versa, and the 'better' configuration is entirely workload-dependent. For latency-bound applications, techniques such as reducing memory channel hops, using faster DRAM timings (lower CAS latency), and increasing cache capacity yield the greatest benefit. For bandwidth-bound applications, widening the memory bus, adding channels (dual/quad/octa-channel configurations), or moving to high-bandwidth memory (HBM) yields far greater returns than reducing latency. Because most real workloads mix both access patterns, modern systems balance these parameters — for example, GPUs prioritize bandwidth for massively parallel throughput workloads, while CPUs prioritize lower latency for control-flow-heavy, branch-dependent code.

3. Hierarchical Memory Organization

No single memory technology can simultaneously offer high speed, large capacity, and low cost — this fundamental trade-off motivates the hierarchical organization of memory into registers, cache, main memory, and secondary storage, arranged from fastest/smallest/most expensive at the top to slowest/largest/cheapest at the bottom.

3.1 Characteristics by Level

Level	Typical Access Time	Typical Capacity	Relative Cost/Bit	Managed By
Registers	< 1 ns (sub-cycle)	Bytes to a few KB	Highest	Compiler / hardware
Cache (L1/L2/L3)	1–20 ns	KB to tens of MB	Very high	Hardware controller
Main Memory (DRAM)	50–100 ns	GBs	Moderate	OS + MMU
Secondary Storage (SSD/HDD)	10 µs (SSD) – 10 ms (HDD)	TBs+	Lowest	OS file system

3.2 Design Rationale — the Locality Principle

This structure works because real programs exhibit locality of reference: at any given time, only a small subset of the total address space is actively used. Placing that active subset in fast, small, expensive storage while the bulk of data resides in slow, large, cheap storage gives the illusion of a memory that is both as fast as the fastest level and as large as the slowest level — at a blended cost close to that of the cheapest level.

3.3 Effective Access Time

The efficiency of the hierarchy can be approximated by a weighted effective access time:

$$T(effective) = H_1 \cdot T_1 + (1 - H_1) \cdot [H_2 \cdot T_2 + (1 - H_2) \cdot T_3 + \ldots]$$

where H_i and T_i are the hit rate and access time at level i. Designers tune the size, associativity, and replacement policy at each level to push the effective access time as close as possible to the fastest level's speed, while keeping overall system cost close to the cheapest level's cost per bit.

3.4 Design Trade-offs

- **Capacity vs. speed:** Larger caches/memories reduce miss rate but increase access latency (due to longer address decode and wire delay) and cost — so each level is deliberately kept small enough to remain fast.
- **Cost vs. speed:** Faster technologies (SRAM in cache, DRAM in main memory) cost significantly more per bit than slower ones (flash, magnetic disk), which is why capacity grows by orders of magnitude at each lower level.
- **Multi-level caching:** Multiple cache levels (L1, L2, L3) exist specifically to interpolate between the extremes — L1 optimizes for speed, L3 optimizes for capacity and shared access across cores.

Overall, an efficient hierarchy is one where each level's hit rate is high enough that the next, slower level is rarely visited — this is precisely why cache design (Question 1) and hierarchy design are two views of the same underlying problem.

4. Virtual Memory and Paging

Virtual memory allows each process to operate as though it has access to a large, contiguous, private address space, independent of the amount of physical RAM actually installed. This is achieved through paging: dividing both virtual and physical address spaces into fixed-size pages/frames and using page tables, managed by the Memory Management Unit (MMU), to translate virtual addresses to physical ones on demand.

4.1 How Virtual Memory Extends Physical RAM

- **Demand paging:** Pages not currently needed are kept on secondary storage (disk/SSD) in a swap area or backing file, and are brought into RAM only when referenced — this is demand paging.
- **Overcommitment:** Multiple processes can be given address spaces larger than physical RAM combined, because inactive pages of each process can be swapped out, enabling efficient multiprogramming.

- **Isolation and protection:** Each process's page table maps it to protected physical frames, preventing one process from reading or corrupting another's memory.

4.2 Page Faults and Their Cost

A page fault occurs when a referenced page is not present in physical memory. The operating system must then: (1) trap into kernel mode, (2) locate the page on secondary storage, (3) select a victim frame to evict if memory is full, (4) write back the victim if it is dirty, (5) read the required page from disk, and (6) update the page table before resuming the process. This sequence can take milliseconds — roughly five to six orders of magnitude slower than a cache hit — making page faults extremely costly to throughput if they occur frequently.

4.3 Page Replacement and Thrashing

When physical memory is full, a page replacement algorithm decides which page to evict. Common policies include:

- **LRU (Least Recently Used):** Evicts the page that has not been used for the longest time; approximates optimal behavior well under typical locality but requires tracking overhead.
- **FIFO:** Evicts pages in arrival order; simple but can suffer from Belady's anomaly (more frames leading to more faults in rare cases).
- **Clock / Second-Chance:** Uses a reference bit to approximate LRU cheaply in hardware, giving a practical middle ground between accuracy and overhead.

If the working set of active processes exceeds available physical memory, the system can enter thrashing — a state where the OS spends more time swapping pages in and out than executing actual instructions, causing throughput to collapse even as CPU utilization appears busy with I/O wait.

4.4 Net Effect on Latency and Throughput

Paging overhead includes the fixed cost of address translation (mitigated by the Translation Lookaside Buffer, or TLB, which caches recent virtual-to-physical mappings) and the variable, much larger cost of actual page faults. A well-tuned system with sufficient RAM, an effective replacement policy, and a high TLB hit rate experiences virtual memory overhead so small it is nearly invisible; an under-provisioned system can see latency spike by several orders of magnitude and throughput drop sharply due to thrashing. Consequently, virtual memory is a powerful abstraction for flexibility and isolation, but its performance is entirely contingent on keeping the page fault rate low relative to the workload's actual working set size.

5. Comparative Evaluation of Memory Technologies

Modern systems draw from a spectrum of memory technologies, each occupying a different point in the latency–bandwidth–capacity–cost–energy trade-off space. The table below compares the major options used in high-performance computing and data-intensive applications.

Technology	Latency	Bandwidth	Capacity	Energy/Endurance
SRAM (on-chip cache)	Sub-nanosecond	Very high (on-die)	KB–MB (limited by die area)	Low energy/access; unlimited endurance

Technology	Latency	Bandwidth	Capacity	Energy/Endurance
DRAM (DDR5)	~50–80 ns	High (tens of GB/s per channel)	GBs, scalable	Moderate energy (refresh overhead); unlimited endurance
HBM (High-Bandwidth Memory)	Comparable to DRAM	Very high (400+ GB/s via wide stacked interface)	GBs (stacked dies)	Efficient per bit moved; unlimited endurance
NVMe SSD (3D NAND)	~10–100 µs	Moderate (GB/s range)	TBs, low cost/bit	Low idle energy; finite program/erase endurance
Storage-Class Memory (e.g., Optane-class)	~1 µs (between DRAM and SSD)	Moderate–high	Hundreds of GB	Byte-addressable, non-volatile; higher endurance than NAND
HDD (magnetic)	~5–10 ms	Low (hundreds of MB/s)	Very large, lowest cost/bit	Low cost but mechanical wear; high seek energy

5.1 Selected Application: Large-Scale Deep Learning Training (GPU-Based)

Deep learning training on large neural networks is used here as the representative high-performance application because it stresses nearly every dimension of the memory hierarchy: it requires extremely high bandwidth for streaming large activation and weight tensors, moderate capacity for holding model parameters and optimizer states, and cannot tolerate high per-access latency during matrix-multiply-heavy compute phases.

5.2 Recommended Memory Hierarchy

- **Level 1 — GPU Registers and L1/Shared Memory (SRAM):** Large multi-level on-die SRAM cache/register files per streaming multiprocessor to hold intermediate tile results during matrix multiplication with sub-nanosecond access.
- **Level 2 — HBM as Main Working Memory:** Stacked HBM directly on the GPU package to feed thousands of parallel cores with the 400+ GB/s bandwidth needed to keep compute units fed during training; its capacity (tens of GB) comfortably holds model weights, gradients, and activation batches for most large models.
- **Level 3 — Host DRAM:** Host DRAM (DDR5) to stage the full dataset, checkpointing buffers, and any model state that overflows GPU HBM (via techniques like offloading in large-model training frameworks), trading slightly higher latency for much larger capacity.
- **Level 4 — NVMe SSD Storage:** NVMe SSD storage-class memory to hold the full training dataset and periodic checkpoints, chosen over HDD for its far higher bandwidth and lower latency, which matters when data loading must keep pace with GPU compute.

5.3 Justification

This configuration is justified because it matches each technology's strength to the corresponding phase of the training pipeline: HBM's bandwidth is the binding constraint for GPU compute utilization (idle GPU cycles waiting on data are the single largest source of wasted throughput in training), so it is prioritized as the primary working memory despite its higher cost per bit. DRAM's larger capacity and acceptable latency make it ideal for staging data just ahead of need, and NVMe SSD's balance of large capacity and reasonable bandwidth makes it far more suitable than HDD for dataset storage, since HDD's millisecond-scale latency would frequently stall data-loading pipelines and starve the GPU. Register and SRAM-level storage, though tiny in capacity, are indispensable because they eliminate redundant HBM traffic for data reused within a compute tile — directly reducing effective memory latency the way cache does in a general-purpose processor (Question 1). Together, this hierarchy delivers the low effective latency, sustained high bandwidth, and sufficient overall capacity that large-scale training demands, while confining the most expensive and power-hungry technology (HBM) to only the tier where its bandwidth is truly the bottleneck.
